

Highlights for Revision

The in-class test will cover the following, please note that this document provides brief highlights, but you should refer the notes for units 1-6 and unit 8 which are on Course Resources for a comprehensive overview.

The topics covered are:

Basics

- Print and Input
- Comments
- Variables and Assignments
- Types
- Arithmetic Operators
- Errors

Branching

- Boolean Expressions
- Logical Operators
- If Statement
- If..else Statement
- If..elif Statement

Loops

- While Statement
- For Statement
- Ranges

Functions

- Function Declaration
- Arguments and Parameters
- Return Statement
- Calling a Function
- Function Composition
- Function Scope

List, Tuples and Dictionaries

Understand which are mutable and which aren't

Lists

- Creating a List
- Basic list methods such as append, insert, remove
- Looping over a list
- Passing a List to a function
- Lists are mutable and ordered (access via an index)

Dictionaries

- Creating a Dictionary
- Accessing, editing and removing dictionary items
- Passing a Dictionary to a function
- Dictionaries are mutable and unordered (access via a key)

Tuples

- Creating a Tuple
- Accessing items
- Tuples are immutable (cannot be changed) and ordered (access via an index)

1. Basics

Print and Input

```
print("Hello World")
```

```
name = input("Please enter your name:")  
print(name)
```

f-strings

```
name = "Sam"  
city = "Derby"  
print(f"{name} lives in {city}.")
```

Comments

```
# this is how you take input from the console  
# this line and the line above are comments  
input_string = input("Please enter your name:")  
print(name)
```

Variables and Assignments

```
# we can assign to variables as follows
x = 10
y = 20

print(x+y)
```

Types

```
# x is of type int
x = 10
print(type(x)) # prints <class 'int'>
```

Four main built-in primitive types `int`, `str`, `float` and `bool`

Arithmetic Operators

```
10 + 3 # 13
10 - 3 # 7
10 * 3 # 30
10**3 # 10*10*10 = 1000
10 + 5 * 3 # do * first. So 10 + 15 = 25
```

Errors

```
10 + "test" # This will result in a TypeError
print(10 # This will result in a SyntaxError
printf("Hello") # This will result in a NameError
```

2. Branching

Boolean Expressions

Operator	Name	Example
<code>==</code>	Equal	<code>x == y</code>
<code>!=</code>	Not equal	<code>x != y</code>
<code>></code>	Greater than	<code>x > y</code>
<code><</code>	Less than	<code>x < y</code>
<code>>=</code>	Greater than or equal to	<code>x >= y</code>
<code><=</code>	Less than or equal to	<code>x <= y</code>

Expression	Result
<code>3 < 5</code>	<code>True</code>
<code>3 > 3</code>	<code>False</code>
<code>6 >= 6</code>	<code>True</code>
<code>3 == 5</code>	<code>False</code>
<code>3 != 5</code>	<code>True</code>

Logical Operators

Boolean expressions can be combined with logical operators to create larger boolean expressions:

Operator	Description	Example
<code>and</code>	Returns <code>True</code> if both statements are <code>True</code>	<code>x < 5 and x < 10</code>
<code>or</code>	Returns <code>True</code> if one of the statements is <code>True</code>	<code>x < 5 or x < 10</code>
<code>not</code>	Reverse the result, returns <code>False</code> if the result is <code>True</code>	<code>not(x < 5 and x < 10)</code>

Order of Precedence

Level	Category	Operators
7(high)	exponent	**
6	multiplication	*, /, //, %
5	addition	+, -
4	relational (comparison)	==, !=, <=, >=, >, <
3	logical	not
2	logical	and
1	logical	or

**

If Statement

```
print("code before if statement")

x = 4

if x < 5:
    print(f"{x} is less than 5")

print("code after if statement")
```

If..else Statement

```
print("code before if statement")

x = 7

if x < 5:
    print(f"{x} is less than 5")
else:
    print(f"{x} is greater than or equal to 5")

print("code after if statement")
```

If..elif Statement

```
if x < y:
    # block of code
    print("x is less than y")
elif x > y:
    # block of code
    print("x is greater than y")
else:
    # block of code
    print("x is equal to y")
```

3. Loops

While Statement

```
i = 1 # Define a variable i and bind the value 1 to it
while i < 6:
    print(i) # print out the value of i
    i += 1   # This increments i by 1. i.e. i = i + 1

print("Program has ended!")
```

```
i = 1
while i < 13:
    j = 1
    while j < 13:
        print(f"{i} x {j} = {i*j}")
        j += 1
    i += 1
```

For Statement

```
for x in ["Citreon", "Ford", 67, True]:
    print(x)
```

```
for i in range(10):
    print(2*i)
```

```
for i in range(1,3):
    for j in range(1,3):
        print(f"{i} x {j} = {i*j}")
```

Ranges

A `range` allows us to create a sequence of numbers using the `range()` function.

```
range(10) # This will create a range of numbers 0,1,2,3,4,5,6,7,8,9
```

```
range(3, 10) # This will create a range of numbers 3,4,5,6,7,8,9
```

```
range(2, 10, 2) # This will create a range of numbers 2,4,6,8
```

4. Functions

Function Declaration

```
def function_name(<formal parameters>):  
    <function body>
```

The easiest way to demonstrate a function is by an example.

Our first function will just add 5 to the input

```
def add_five(x):  
    return x + 5
```

Note that the `return` keyword is how you return (output) something from a function. Here the function `add_five()` returns the result of adding 5 to the input `x`.

Arguments and Parameters

It is important to understand the difference between the parameters of a function and arguments. These words are often confused or used interchangeably.

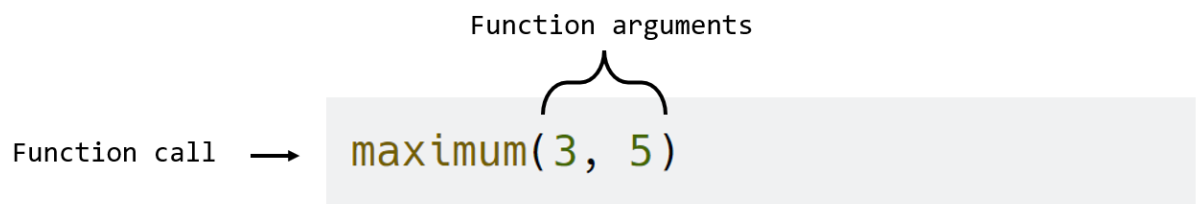
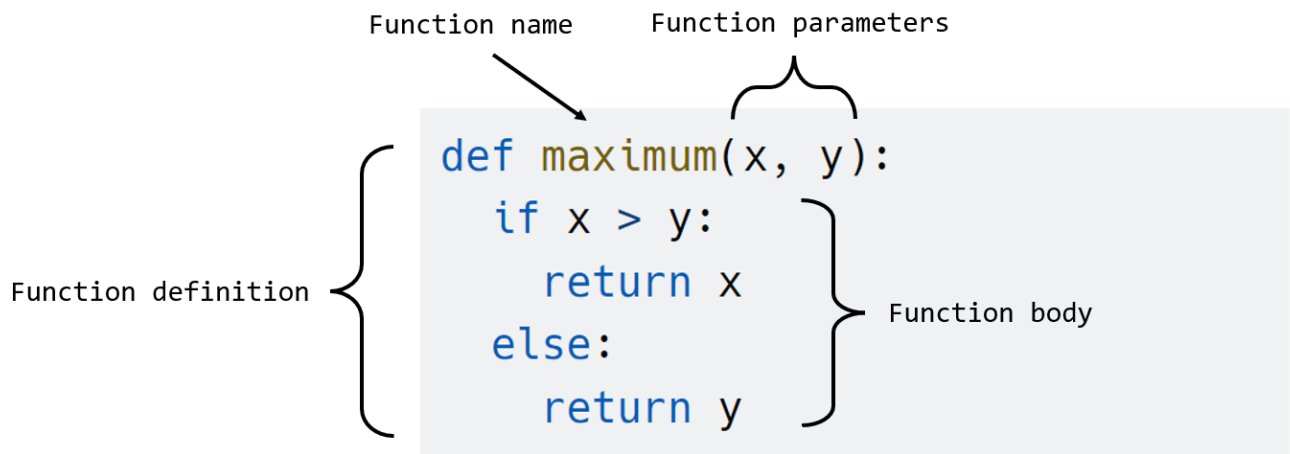
Parameters

Parameters are the names that are in the function definition in between the parentheses `()`.

Arguments

Arguments are the names that are used in the function call.

The following image shows both the function definition and the function call.



Calling a Function

```
def add(x, y):  
    return x + y  
  
result = add(3, 13)  
print(result) # prints 16
```

The second returns the maximum of the two inputs.

```
def maximum(x, y):  
    if x > y:  
        return x  
    else:  
        return y  
  
result = maximum(3, 5)  
print(result) # prints 5
```

You will note that this function has two uses of `return` for the two possible cases.

```
def print_hello(name):  
    print(f"Hello {name}")  
  
# call the function and bind the return value to a  
a = print_hello("Jimi")  
print(a)
```

You will notice that it outputs,

```
Hello Jimi  
None
```

Function Composition

```
def add5(x):  
    return x + 5  
  
def mul2(x):  
    return 2*x  
  
x = 10      # binds 10 to x  
x = add5(x) # binds 15 to x, i.e. 10 + 5 = 15  
x = mul2(x) # binds 30 to x, i.e. 2*15 = 30  
print(x)    # prints 30
```

Function Scope

```
def f(x):  
    # variables defined here are only available to f() - local  
    # we also do not have access to variables created outside of f()  
    y = 1  
    x = x + y  
    print(f"x = {x}")  
    return x  
  
# main body of code starts here  
x = 3  
y = 3  
z = f(x)  
print(f"z = {z}")  
print(f"x = {x}")  
print(f"y = {y}")
```

5. List, Tuples and Dictionaries

Understand which are mutable and which aren't.

Lists

1. Ordered (Indexed by number)
2. Mutable (changeable)

By **ordered** we mean that the tuple has an order, we use this to access items.

By **mutable** we mean that the items in the list can be changed and we can add/remove items

```
list_one = [1,2,"Sam"]
print(list_one) # prints [1,2,"Sam"]

# Access item at index 2 (third item "Sam")
print(list_one[2])

# Change item at index 1 to "Brad"
list_one[1] = "Brad"
print(list_one) # prints [1,"Brad","Sam"]

# Add "Wayne" to end of list
list_one.append("Wayne")
print(list_one) # prints [1,"Brad","Sam","Wayne"]

# Remove the item "Brad"
list_one.remove("Brad")
print(list_one) # prints [1,"Sam","Wayne"]

# Remove the item at index 2
item_removed = list_one.pop(2)
print(item_removed) # prints "Wayne"
print(list_one) # prints [1,"Sam"]

# we can loop over the list
for item in list_one:
    print(item)

# prints
# 1
# Sam

# we can also do this with a while Looping
i = 0
while i < len(list_one):
    print(list_one[i])
    i += 1

# we can also use functions on lists
num_list = [1,2,3,4]
```

```
print(len(num_list))  
print(sum(num_list))
```

Dictionaries

1. Unordered (Indexed by Key)
2. Mutable (changeable)

By **unordered (indexed by key)** we mean that each value that we store in the dictionary has a unique key that allows you to access it.

By **mutable** we mean that the items in the dictionary can be changed and we can add/remove key:value pairs.

```
student_dict = {
    1123425: "Count Dracula",
    1425243: "Frankenstein's Monster",
    1512414: "The Mummy"
}
# stores student names by their id.
# Here key:value pair is id: name

# Here is another example using different types as keys and values.
test_dict = {
    1: "Hello World",
    "Test": False,
    "Another Key": [1,2,3,4]
}

### Access Dictionary Items
print(student_dict[1123425]) # prints "Count Dracula"

### Change Dictionary Items
student_dict[1123425] = "Jack Skeleton" # changes the value to "Jack Skeleton"
print(student_dict)
# prints
# {1123425: 'Jack Skeleton',
# 1425243: 'Frankenstein's Monster',
# 1512414: 'The Mummy'}
```

```
### Add Dictionary Items
# adds the key:value pair to the dictionary
student_dict[1234567] = "Casper The Ghost"
print(student_dict)
# prints
# {1123425: 'Jack Skeleton',
# 1425243: 'Frankenstein's Monster',
# 1512414: 'The Mummy',
# 1234567, 'Casper The Ghost'}
```

```
### Remove Dictionary Items
del student_dict[1425243] # removes the 1425243: "Frankenstein's Monster"
print(student_dict)
# prints
# {1123425: 'Jack Skeleton', 1512414: 'The Mummy', 1234567, 'Casper The Ghost'}
```


Looping Through a Dictionary

You can loop through a dictionary in a number of ways.

If you want to loop through the keys you can do the following.

```
student_dict = {
    1123425: "Count Dracula",
    1425243: "Frankenstein's Monster",
    1512414: "The Mummy"
}
for key in student_dict.keys():
    print(key)

# prints...
# 1123425
# 1425243
# 1512414

# or you can get the values using the keys
for key in student_dict.keys():
    print(student_dict[key])

# prints...
# "Count Dracula"
# "Frankenstein's Monster"
# "The Mummy"

# You can also use this to get the keys and values:

for key in student_dict.keys():
    print(f"Key = {key}, Value = {student_dict[key]}")

# prints...
# Key = 1123425, Value = Count Dracula
# Key = 1425243, Value = Frankenstein's Monster
# Key = 1512414, Value = The Mummy
```

Another way, perhaps the most common is using the `items()` method.

```
for k, v in student_dict.items():
    print(f"Key = {k}, Value = {v}")
```

Here `items` returns a list (actually a type called `dict_items`) of (`key`, `value`) pairs that you can then bind to the variables `k` and `v` for each iteration of the loop.

Tuples

1. Ordered
2. Immutable (unchangeable)

By **ordered** we mean that the tuple has an order and it cannot be changed.

By **immutable** we mean that the tuple cannot be added to, removed from and items cannot be changed.

You define a tuple in much the same way as a list, but with round brackets, not square.

```
car_tuple = ("Audi", "Mercedes", "Ferrari")
```

You can access tuple items using indexing, e.g.

```
print(car_tuple[1]) # prints the 2nd item of the tuple, "Mercedes"
```

You can also use slicing and negative indexing as per lists.

```
print(car_tuple[0:2]) # prints the 1st and 2nd items of the tuple, ("Audi", "Mercedes")
print(car_tuple[-1]) # prints the last item of the tuple, "Ferrari"
```

You can loop through a tuple just like a list using a **for** loop.

```
car_tuple = ("Audi", "Mercedes", "Ferrari")

for car in car_tuple:
    print(car)

# prints..
# Audi
# Mercedes
# Ferrari
```